

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Факультет систем управления и робототехники

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ
по дисциплине «Прикладный Искусственный Интеллект»
Практическое задание № 2. «Нейронные сети»

Выполнила: Нгуен Кхань Нгок – R3335
Преподаватель: Евстафьев Олег Александрович

Санкт Петербург
2024

1. Цель работы

Прогнозирование цены на жилье с помощью нейросетевой регрессионной модели. Необходимо по имеющимся данным о ценах на жильё предсказать окончательную цену каждого дома с учетом характеристик домов с использованием нейронной сети.

Описание набора данных содержит 80 классов (набор переменных) классификации оценки типа жилья, и находится в файле `data_description.txt`.

В работе требуется дополнить раздел «Моделирование» в подразделе «Построение и обучение модели» создать и инициализировать последовательную модель нейронной сети с помощью фреймворков тренировки нейронных сетей как: Torch или Tensorflow.

Скомпилировать нейронную сеть выбрав функцию потерь и оптимизатор соответственно. Оценить точность полученных результатов. Вывести предсказанные данные о продаже.

2. Описание выполненной работы

Программа работы

3. Требования при выполнении работы

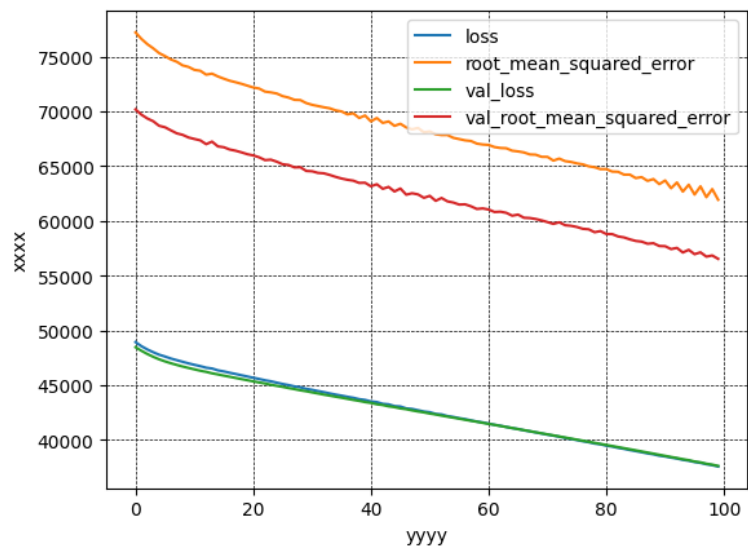
- Выведите отчет нейросетевой регрессионной модели, для прогнозирования цены на жилье.
- Подберите разные комбинации гиперпараметров таким образом, чтобы получить лучший результат на тестовом наборе данных.
- Попробуйте использовать разное количество нейронов на входном слое. Опишите достигнутый результат.
- Добавьте в нейронную сеть скрытый слой с разным количеством нейронов.
- Используйте разное количество эпох. Опишите достигнутый результат.
- Используйте разные размеры мини-выборки (`batch_size`). Опишите достигнутый результат.
- Попробуйте использовать разные значения оптимизатора `'optimizers'` и функции потерь `'loss'`. Сравните полученные результаты.

4. Результат работы

4.1. Первая модель

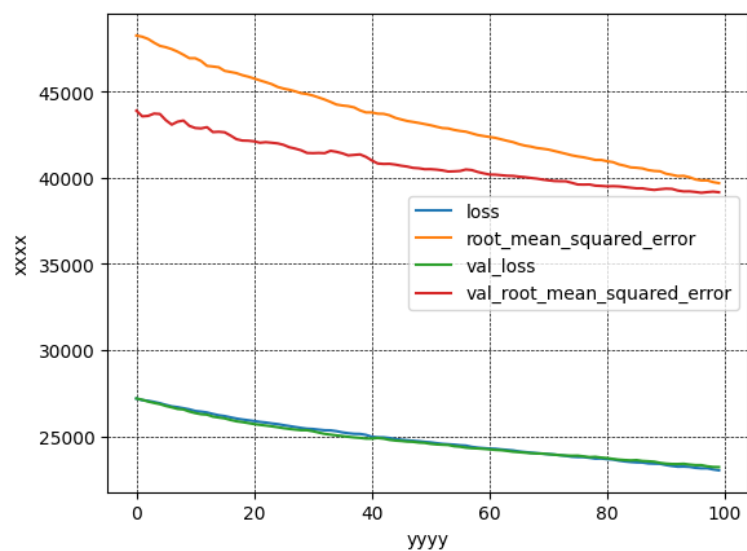
```
model = Sequential([
    layers.Input(shape=[75,1]),
    layers.Dense(128, activation='relu'),
    layers.Dense(64, activation='relu'),
    layers.Dense(32, activation='relu'),
    layers.Dense(1)
])
# Для обеспечения воспроизводимости результатов устанавливается функция seed
tf.random.set_seed(40)
```

4.1.1. epoch = 145, batch_size = 199



loss: 37864.5391 - root_mean_squared_error: 55980.1172

4.1.2. epoch = 100, batch_size = 199

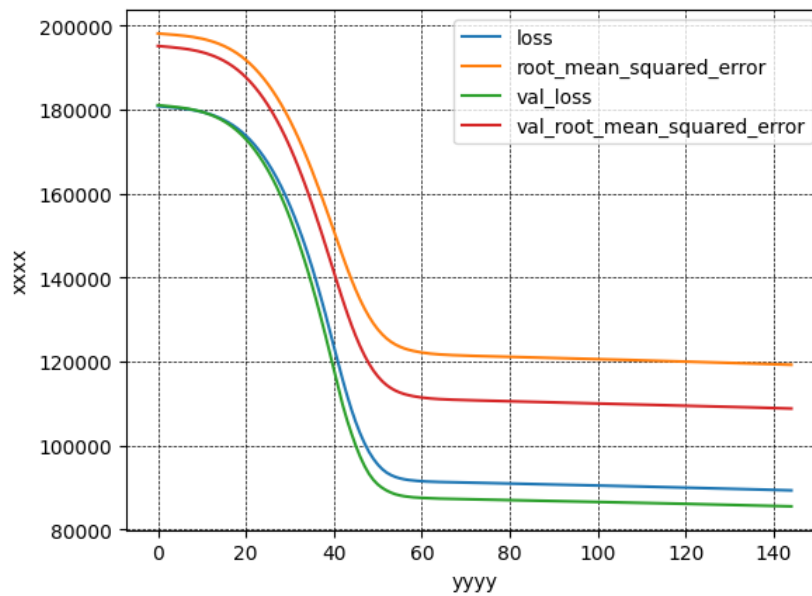


loss: 25679.4023 - root_mean_squared_error: 46022.4727

4.2. Вторая модель

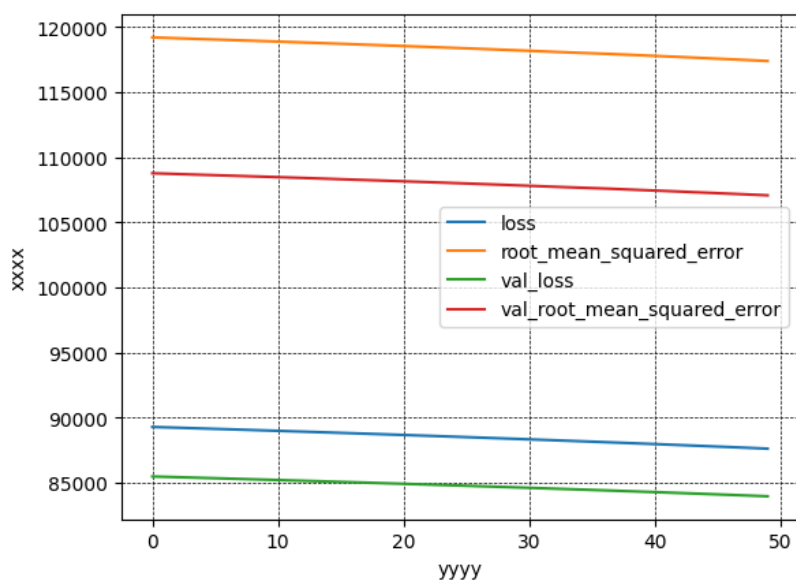
```
model = Sequential([
    layers.Input(shape=[75,]),
    layers.Dense(64, activation='relu'),
    layers.Dense(32, activation='relu'),
    layers.Dense(1)
])
# Для обеспечения воспроизводимости результатов устанавливается функция seed
tf.random.set_seed(40)
```

4.2.1. epoch = 145, batch_size = 199



loss: 46034.7109 - root_mean_squared_error: 66513.2969

4.1.2. epoch = 50, batch_size = 199

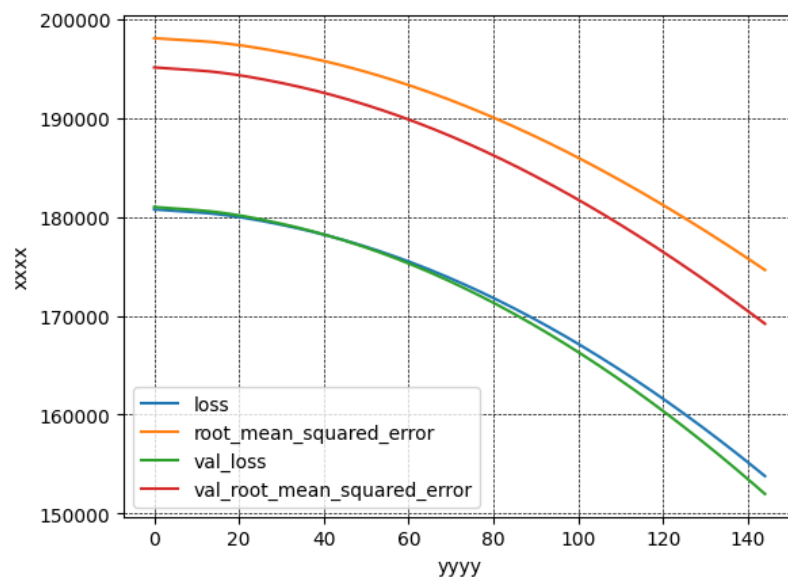


loss: 83554.2891 - root_mean_squared_error: 108253.2109

4.3. Третья модель

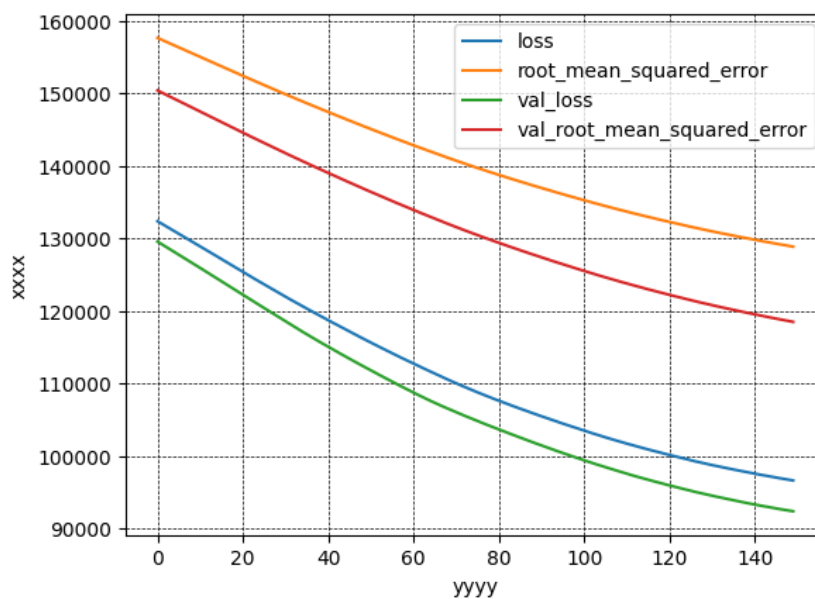
```
model = Sequential([
    layers.Input(shape=[75,]),
    layers.Dense(32, activation='relu'),
    layers.Dense(1)
])
# Для обеспечения воспроизводимости результатов устанавливается функция seed
tf.random.set_seed(40)
```

4.3.1. epoch = 145, batch_size = 199



loss: 152719.3750 - root_mean_squared_error: 169517.5156

4.3.2. epoch = 150, batch_size = 200

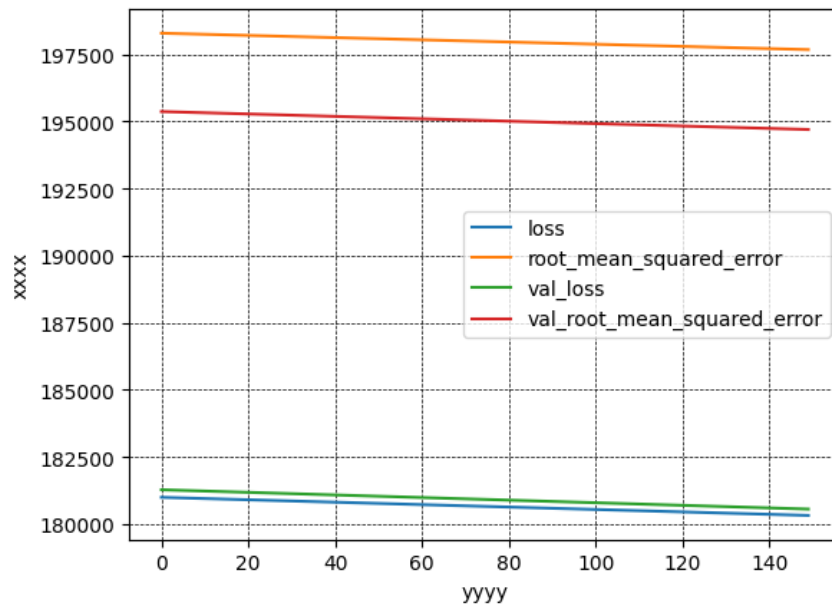


loss: 93727.5859 - root_mean_squared_error: 119538.3438

4.4. Четвертая модель

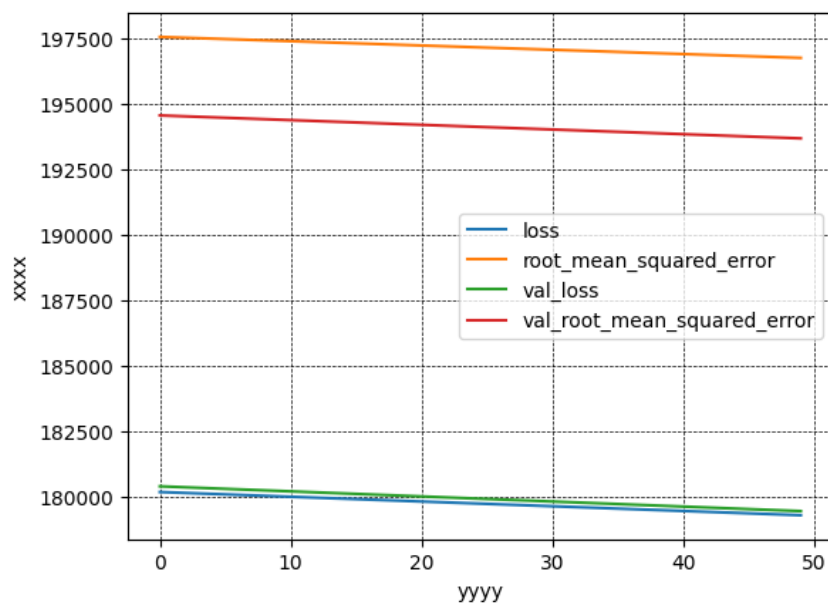
```
model = Sequential([
    layers.Input(shape=[75,]),
    layers.Dense(1)
])
# Для обеспечения воспроизводимости результатов устанавливается функция seed
tf.random.set_seed(40)
```

4.3.1. epoch = 150, batch_size = 200



loss: 181186.5781 - root_mean_squared_error: 194787.1719

4.3.2. epoch = 150, batch_size = 50



loss: 177189.7188 - root_mean_squared_error: 191115.2812

Итоги работы:

Лучший результат: `root_mean_squared_error: 46022.4727`

Этот результат достигается при следующих значениях параметров нейронной сети:

- Количество нейронов в скрытом слое: 128 - 64 - 32
- Количество скрытых слоев: 3
- Функции активации: ReLu
- Количество эпох: 100
- Размер мини-выборки: 199
- Оптимизатор: adam

Вопросы:

Как выше перечисленные параметры влияют на полученный вами результат?

- По мере увеличения количества эпох потери обычно уменьшаются, если модель хорошо обучается. Однако, когда эпох слишком много, может произойти переобучение.
- ReLU эффективен для нелинейных зависимостей, ускоряет обучение
- Эпохи — это число проходов через весь датасет. 100 эпох позволяют модели достичь хорошей сходимости, но если их больше, возможен риск переобучения.
- Большой batch ускоряет обучение, но уменьшает детализацию градиента.
- Adam автоматически адаптирует скорость обучения, что делает его хорошим выбором для большинства задач. Он стабилен и эффективен.

Что такое эпоха (Epoch)? В чем отличие от итерации (Iteration)?

- Эпоха (epoch) — это полный цикл обучения, в ходе которого модель один раз проходит весь набор данных.
- Итерация — это этап процесса обучения, когда модель обновляет веса после обработки небольшой группы данных (пакета)

Разница:

- Эпоха включает в себя множество итераций.
- Количество итераций в эпоху зависит от размера набора данных и размера пакета.

Что такое функция активации? Какие вам известны? Как и зачем используются в нейронной сети?

- Функция активации — это фрагмент программного кода, добавляемый в искусственную Нейронную сеть, чтобы помочь ей изучить сложные закономерности данных. Функция активации решает, что должно быть запущено в следующий нейрон. Она принимает выходной сигнал из предыдущей ячейки и преобразует его в некоторую форму, которую можно использовать.

- ReLu ($\text{Input}(x) = \max(0, x)$) представляет собой функцию, которая возвращает значение x , если x положительно, и 0 в противном случае. Это наиболее универсальная функция, показывающая хорошие результаты в подавляющем большинстве случаев.
- Softmax: Используется для многоклассовой классификации, преобразует выходы в вероятности.

Что такое MSE (Mean Squared Error) - Средняя квадратичная ошибка? Что такое MAE (Mean Absolute Error)? Для чего используются.

- MSE вычисляет среднее значение ошибок между фактическим значением и прогнозируемым значением, а затем увеличивает большие ошибки, возводя их в квадрат.
- MAE — это среднее значение абсолютных отклонений между предсказанными и истинными значениями. В отличие от MSE, MAE не возводит ошибки в квадрат и более "устойчива" к выбросам.
- MSE и MAE – функции ошибок, которые используются для оценки алгоритма работающие по следующим формулам:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

Mean
Error
Squared

$$MAE = \frac{1}{n} \sum |y - \hat{y}|$$

Divide by the total number of data points
Actual output value
Predicted output value

Sum of
The absolute value of the residual

Где y_i фактический ожидаемый результат и $\hat{y}_i$ это прогноз модели

